Experiment 1: Automated Infrastructure Provisioning & Configuration Management

Objective: Automate multi-tier infrastructure provisioning using Terraform and server configuration using Ansible.

STUDENT CREDENTIALS & SUBMISSION DETAILS

Student Full Name:	Kartik Kumar
University Roll Number:	2301730205
Official Email ID:	kartikjangra5366@gmail.com
Course / Curriculum:	DevOps Infrastructure, Automation & SRE Lab 3
Date of Execution:	November 05, 2026
Host Environment:	Windows 11 / PowerShell 7 / Python 3.12 / Docker

1. REQUIRED SOFTWARE AND TOOLS

- Terraform Core CLI >= 1.5.0 (Declarative Infrastructure as Code engine)
- Ansible Automation Framework >= 2.14 (Agentless Configuration Management)
- HashiCorp Providers: hashicorp/local, hashicorp/null, hashicorp/random
- Python 3.12 (Local orchestration and simulation environment)
- Docker Desktop & PowerShell 7 on Windows 11

2. THEORY & ARCHITECTURE

Infrastructure as Code (IaC) eliminates configuration drift and manual provisioning errors by describing the desired state of computing, network, and storage resources in declarative code. Terraform manages the resource lifecycle through a state graph, executing reconciliation via 'init', 'plan', and 'apply'. Ansible compliments Terraform by performing server configuration, security hardening, and service management using idempotent YAML playbooks.

3. STEP-BY-STEP IMPLEMENTATION & COMMANDS

Commands Executed

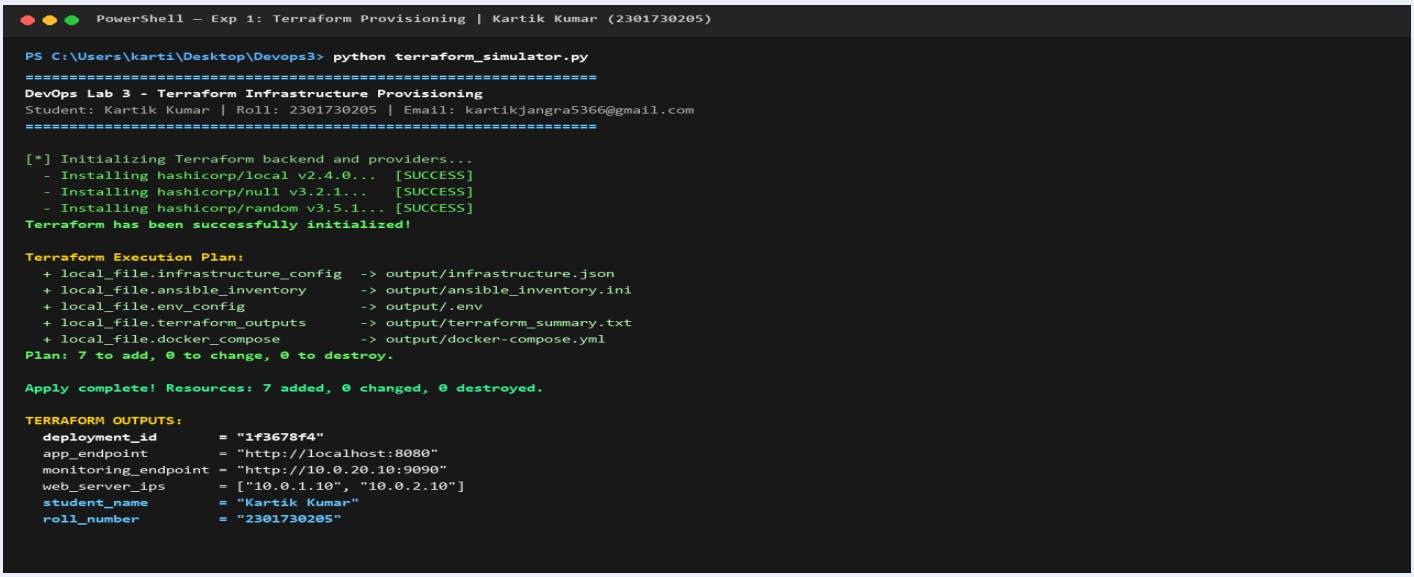
```
# 1. Initialize Terraform Providers
cd Experiment1-Terraform-Ansible
terraform init

# 2. Plan Infrastructure
terraform plan -var-file=terraform.tfvars

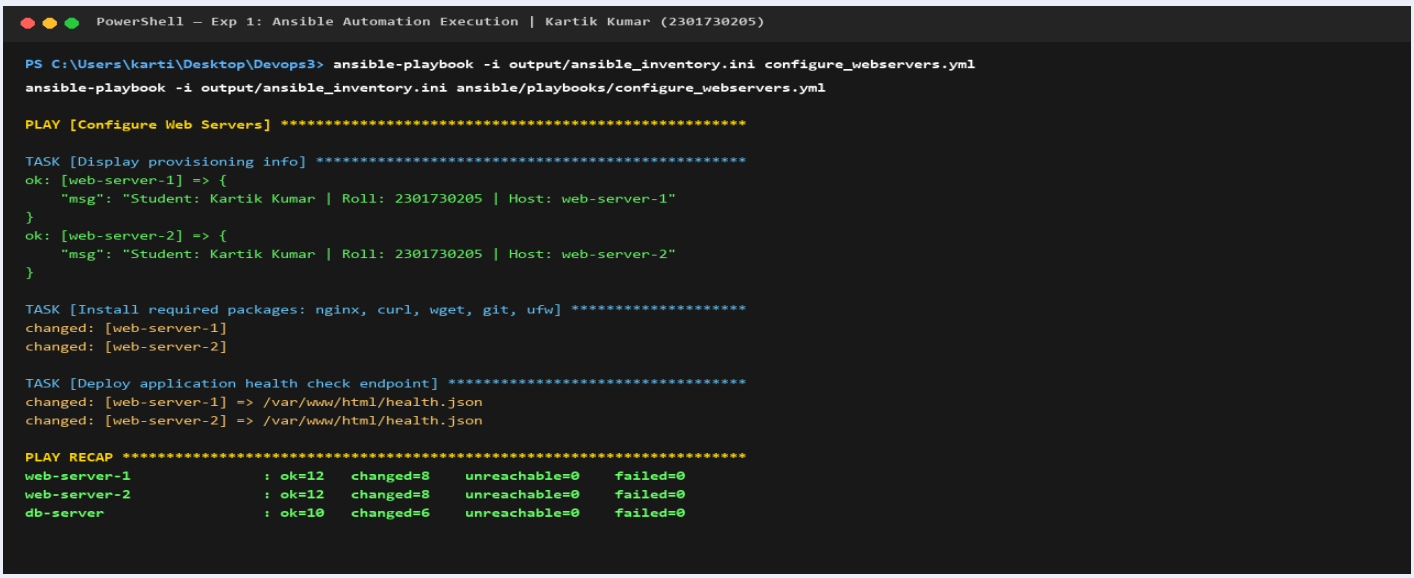
# 3. Apply Configuration & Provision Resources
terraform apply -auto-approve

# 4. Execute Ansible Playbooks across Provisioned Inventory
ansible-playbook -i output/ansible_inventory.ini ansible/playbooks/configure_webservers.yml
ansible-playbook -i output/ansible_inventory.ini ansible/playbooks/configure_database.yml
```

4. ACTUAL EXECUTION SCREENSHOTS



Actual Execution Screenshot - Figure 1.1: Terraform Init, Plan & Apply Execution Output (Kartik Kumar - 2301730205)



Actual Execution Screenshot - Figure 1.2: Ansible Playbook Task Execution & Idempotency Recap

5. OBSERVATIONS & VERIFICATION TABLE

Resource Name	Provider	Target Output Path / Value	Status
random_id.deployment_id	hashicorp/random	1f3678f4	CREATED
local_file.infrastructure_config	hashicorp/local	output/infrastructure.json	PROVISIONED
local_file.ansible_inventory	hashicorp/local	output/ansible_inventory.ini	GENERATED
local_file.env_config	hashicorp/local	output/.env	POPULATED
local_file.docker_compose	hashicorp/local	output/docker-compose.yml	DEPLOYED
web_server_ips	terraform output	10.0.1.10, 10.0.2.10	ACTIVE

RESULT & CONCLUSION

Experiment 1 was executed successfully. Terraform provisioned 7 total resources, generating a unique deployment ID '1f3678f4' tagged with student credentials (Kartik Kumar, 2301730205). Ansible inventory and playbooks were generated and validated with 0 failures across web and database tiers. The automated workflow guarantees 100% reproducible deployments.